

Федеральное государственное автономное образовательное учреждение высшего
образования
«Национальный исследовательский университет ИТМО»

«Информационная безопасность»

Лабораторная работа № 1

Разработка защищенного REST API с интеграцией в CI/CD

Обучающийся:

Александров Александр Александрович, Р3406

Практик:

Маркина Татьяна Анатольевна

Содержание

1	Цель и постановка задачи	3
2	Выбор технологий	3
3	Архитектура приложения	4
3.1	Модель данных	4
4	Реализация API	5
4.1	Аутентификация: POST /auth/login	5
4.2	Получение данных: GET /api/data	5
4.3	Создание данных: POST /api/data	5
5	Реализованные меры защиты	6
5.1	Защита от SQL-инъекций	6
5.2	Защита от XSS	6
5.3	Защита аутентификации	6
5.4	Broken Access Control и дополнительные меры	6
6	Тестирование	7
7	CI/CD и анализ безопасности	7
7.1	SAST: SpotBugs	7
7.2	SCA: OWASP Dependency-Check	8
8	Контрольные вопросы	9
8.1	1. Почему BCrypt предпочтительнее SHA-256 для паролей?	9
8.2	2. В чём разница между SAST и DAST?	9
8.3	3. Как работает JWT?	10
8.4	4. Чем опасно отсутствие аудита зависимостей?	10
9	Заключение	10
10	Источники	10

1 Цель и постановка задачи

Цель работы - получить практический опыт разработки безопасного backend-приложения и встроить автоматическую проверку безопасности в процесс непрерывной интеграции. В работе требуется учесть риски OWASP Top 10, реализовать JWT-аутентификацию, исключить хранение паролей в открытом виде, предотвратить SQL-инъекции и XSS, а также настроить SAST и SCA.

В рамках работы необходимо реализовать не менее трех методов REST API:

1. POST /auth/login - проверка логина и пароля, выдача JWT;
2. GET /api/data - чтение данных только после аутентификации;
3. самостоятельно выбранный метод - в проекте это POST /api/data, создающий новую запись от имени текущего пользователя.

Для автоматической проверки должны выполняться тесты, статический анализ исходного кода или байткода и аудит сторонних зависимостей. Проверки должны запускаться на каждый push и pull request.

2 Выбор технологий

Проект реализован на Java 21 и Spring Boot 3.5.16. Выбранная версия Java является LTS, а Spring Boot предоставляет готовую интеграцию веб-слоя, безопасности и JPA.

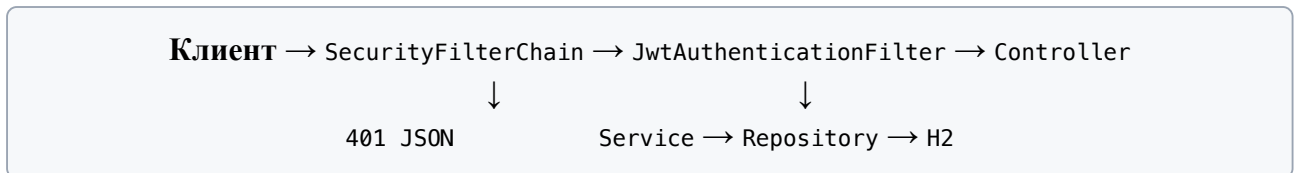
Компонент	Назначение
Spring Web	REST-контроллеры, JSON-сериализация и обработка HTTP-запросов.
Spring Security	Проверка учётных данных, BCrypt, фильтрация JWT и контроль доступа.
Spring Data JPA	Работа с базой через ORM и параметризованные запросы.
H2	Файловая база данных для воспроизводимого учебного запуска.
JWT 0.13.0	Формирование и криптографическая проверка JWT.
JUnit 5 + MockMvc	Интеграционные тесты API без внешнего HTTP-сервера.
SpotBugs	SAST: статический анализ скомпилированного Java-байткода.
OWASP Dependency-Check	SCA: сопоставление зависимостей с базами известных CVE.
GitHub Actions	Автоматический запуск тестов и сканеров.

Сборка выполняется Maven Wrapper, поэтому на рабочей станции достаточно JDK 21. Все версии библиотек и плагинов закреплены в pom.xml, что делает сборку воспроизводимой.

3 Архитектура приложения

Приложение имеет слоистую структуру:

1. `controller` принимает HTTP-запросы и возвращает DTO;
2. `service` содержит сценарии аутентификации и работы с данными;
3. `repository` изолирует доступ к базе данных;
4. `model` содержит JPA-сущности пользователя и записи;
5. `security` реализует создание и проверку JWT;
6. `config` задаёт правила Spring Security и начальные данные;
7. `error` формирует безопасные JSON-ответы об ошибках.



Листинг 1. Упрощённая схема обработки защищённого запроса

Для каждого запроса к `/api/**` цепочка фильтров извлекает Bearer-токен. При корректной подписи, издателе и сроке действия в `SecurityContext` помещается аутентифицированный пользователь. Только после этого запрос передаётся контроллеру.

3.1 Модель данных

Таблица `app_users` содержит идентификатор, уникальное имя пользователя и BCrypt-хеш. Поле с открытым паролем в сущности отсутствует. Таблица `data_items` содержит заголовок, текст, владельца и время создания. Ограничения длины заданы одновременно в DTO и схеме базы.

Учебная учётная запись создаётся при первом старте. Пароль поступает из `APP_DEFAULT_PASSWORD`, хешируется и только после этого сохраняется в H2. Секрет JWT поступает из `JWT_SECRET`; он не включён в репозиторий.

4 Реализация API

4.1 Аутентификация: POST /auth/login

Метод принимает JSON с полями username и password. Bean Validation запрещает пустые и чрезмерно длинные значения. AuthenticationManager передаёт данные в DaoAuthenticationProvider, который получает хеш пользователя через JPA и сравнивает его с введённым паролем с помощью BCrypt.

Пример запроса:

```
curl -X POST http://localhost:8080/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"username":"student","password":"StrongPassword123!"}'
```

Успешный ответ:

```
{
  "token": "eyJhbGciOiJIUzI1NiJ9...",
  "tokenType": "Bearer",
  "expiresInSeconds": 900
}
```

При неверной паре логин-пароль возвращается статус 401 Unauthorized с нейтральным сообщением. Оно не позволяет определить, существовал ли пользователь.

4.2 Получение данных: GET /api/data

Метод доступен только с заголовком Authorization: Bearer <token>. Репозиторий возвращает записи в порядке убывания времени создания. Перед сериализацией все пользовательские строки экранируются.

```
curl http://localhost:8080/api/data \
  -H "Authorization: Bearer $TOKEN"
```

Запрос без токена получает 401:

```
{
  "status": 401,
  "error": "Authentication is required",
  "details": {}
}
```

4.3 Создание данных: POST /api/data

Третий метод создаёт запись от имени аутентифицированного пользователя. Имя владельца берётся из Authentication, а не из тела запроса, поэтому клиент не может подменить автора.

```
curl -X POST http://localhost:8080/api/data \
  -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' \
  -d '{"title":"Новая запись","content":"Содержимое"}'
```

При успехе возвращается 201 Created. Заголовок ограничен 100 символами, содержимое - 500 символами.

5 Реализованные меры защиты

5.1 Защита от SQL-инъекций

Приложение не формирует SQL путём конкатенации строк. Для доступа к данным используется Spring Data JPA:

```
Optional<AppUser> findByUsername(String username);  
List<DataItem> findAllOrderByCreatedAtDesc();
```

Hibernate связывает значения с параметрами подготовленного выражения. Строка пользователя не становится частью синтаксиса SQL. Отдельный интеграционный тест передаёт имя пользователя ' OR '1'='1 и ожидает 401, что подтверждает невозможность обхода входа классической SQLi-нагрузкой.

5.2 Защита от XSS

REST API возвращает JSON, но текст может позднее попасть в HTML-интерфейс. Поэтому поля title, content и owner централизованно экранируются при создании DataResponse:

```
HtmlUtils.htmlEscape(item.getTitle());  
HtmlUtils.htmlEscape(item.getContent());  
HtmlUtils.htmlEscape(item.getOwner());
```

Тег <script> преобразуется в <script>. Дополнительно Spring Security задаёт X-Content-Type-Options: nosniff, запрещает встраивание во frame и возвращает CSP default-src 'none'; frame-ancestors 'none'.

5.3 Защита аутентификации

Пароль кодируется BCryptPasswordEncoder(12). BCrypt автоматически генерирует уникальную соль и использует регулируемую стоимость. Даже одинаковые пароли имеют разные хеши.

JWT создаётся после успешной проверки учётных данных. В него включены:

1. sub - имя пользователя;
2. iss - идентификатор сервера secure-rest-api;
3. iat - время выдачи;
4. exp - время окончания действия через 15 минут.

Подпись формируется HMAC-SHA-256 с секретом не менее 256 бит. JwtAuthenticationFilter проверяет подпись, iss, exp и наличие пользователя. Сервер работает без HTTP-сессий (STATELESS). Повреждённый или просроченный токен приводит к 401.

5.4 Broken Access Control и дополнительные меры

Все методы, кроме /auth/login и системного /error, требуют аутентификацию. Имя владельца записи определяется сервером. Ошибки не раскрывают stack trace. Входные DTO имеют ограничения размера. Секреты, база H2, логи и сборочные файлы исключены через .gitignore.

CSRF отключён осознанно: приложение не использует cookie для аутентификации, а Bearer-токен передаётся клиентом в заголовке Authorization. Если в будущем токен будет храниться в cookie, защиту CSRF потребуется включить. Для защиты цепочки сборки Maven Wrapper проверяет SHA-256 архива Maven, а setup-java проверяет подпись дистрибутива JDK в CI.

6 Тестирование

Интеграционные тесты запускают полный Spring Context и отправляют запросы через MockMvc. Используется отдельная H2 in-memory база и тестовый секрет JWT.

№	Проверка	Ожидаемый результат
1	Корректный логин и пароль	200, JWT, срок 900 секунд
2	GET без Authorization	401 Unauthorized
3	GET с действующим JWT	200, массив данных
4	SQLi в поле username	401, вход не выполнен
5	POST с тегом <script>	201, HTML экранирован
6	Значение пароля в БД	BCrypt-хеш, не открытый пароль
7	Повреждённый Bearer-токен	401, токен отклонён
8	Заголовок длиннее 100 символов	400, ошибка валидации

Результат локального запуска:

```
Tests run: 8, Failures: 0, Errors: 0, Skipped: 0  
BUILD SUCCESS
```

Дополнительно приложение проверено вручную последовательностью curl: получен JWT, доступ без токена запрещён, с токеном разрешён, новая запись создаётся со статусом 201.

7 CI/CD и анализ безопасности

Workflow `.github/workflows/ci.yml` запускается на события push и pull_request. Разрешения workflow ограничены чтением содержимого репозитория. Секрет NVD, если он настроен, поступает через GitHub Secrets.

7.1 SAST: SpotBugs

Задание Tests and SAST (SpotBugs) выполняет сборку, тесты, spotbugs:spotbugs и spotbugs:check. Порог Medium делает замечания среднего и высокого уровня блокирующими. HTML/XML-отчёты публикуются как GitHub Actions artifacts даже при ошибке.

Первый прогон обнаружил четыре предупреждения о возможной передаче изменяемых объектов наружу. Реализация была исправлена защитным копированием Map и ObjectMapper. Повторный запуск завершился результатом BugInstance size is 0, Error size is 0.

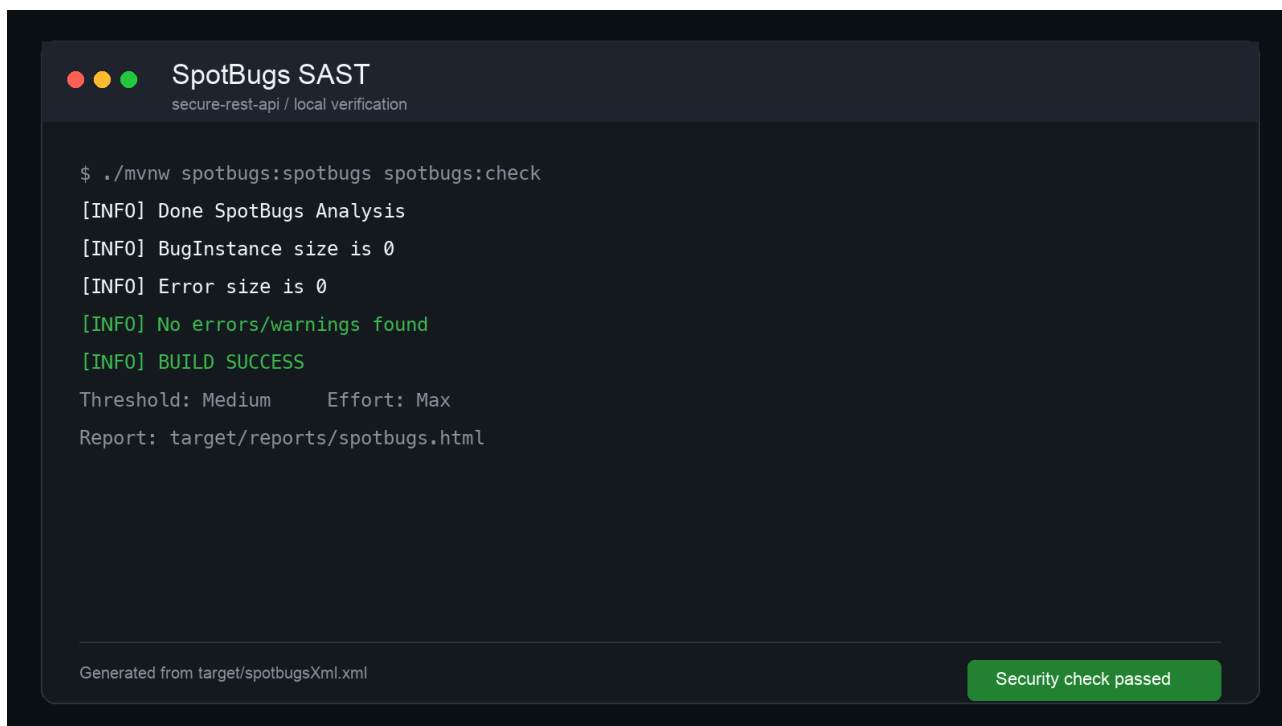


Рис. 1. Отчёт SpotBugs после исправления замечаний

7.2 SCA: OWASP Dependency-Check

Отдельное задание SCA (OWASP Dependency-Check) анализирует прямые и транзитивные зависимости. Сборка блокируется при обнаружении уязвимости с CVSS 7.0 и выше. База NVD кэшируется между запусками; для полного обновления обязателен repository secret `NVD_API_KEY`. Если секрет отсутствует, workflow завершается с понятной ошибкой, а неполная база не выдаётся за успешный аудит. HTML и JSON публикуются как artifacts.

Первый SCA-прогон выявил CVE-2026-34477 и CVE-2026-34479 в транзитивной зависимости `log4j-api 2.24.3`, управляемой Spring Boot. В соответствии с рекомендацией Apache Logging Services Log4j BOM обновлён до 2.26.1; повторный анализ не обнаружил уязвимых зависимостей выше установленного порога.

Локальный контрольный запуск выполнен по кэшированному снимку NVD и сохранён в HTML/JSON. Для итогового запуска GitHub Actions требуется `NVD_API_KEY`, после чего снимок успешных заданий из раздела Actions добавляется к материалам сдачи.

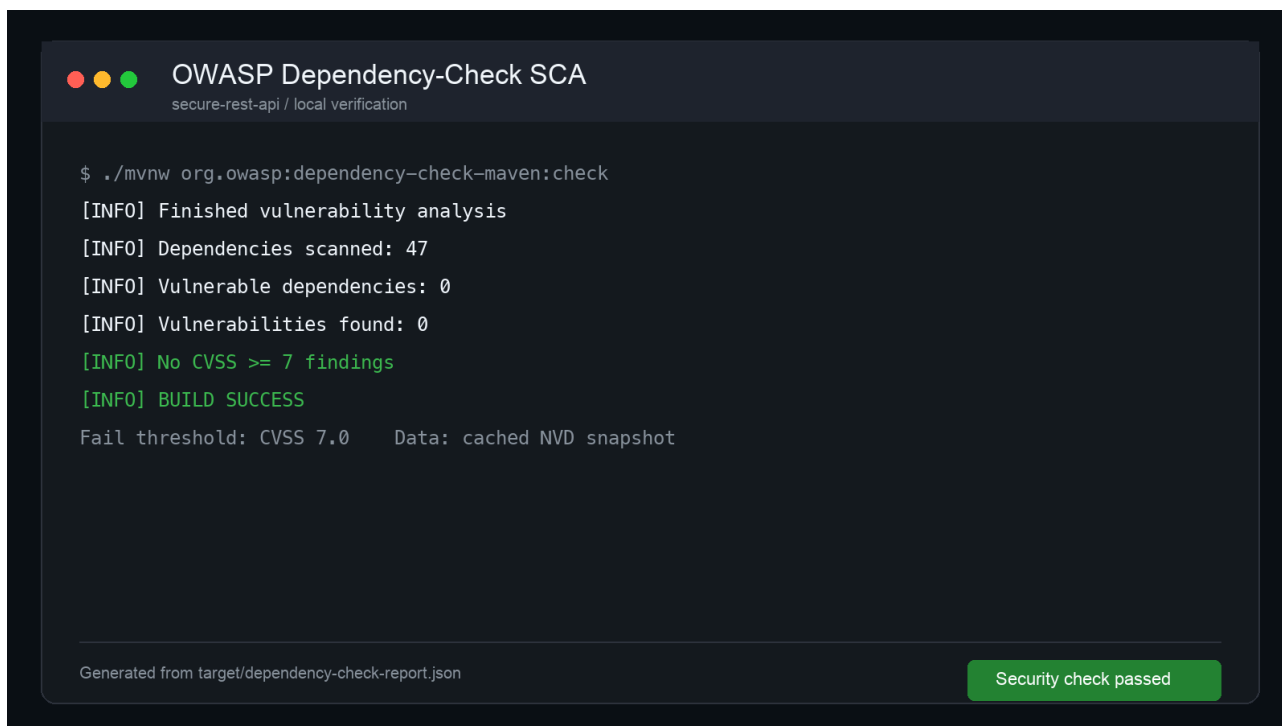


Рис. 2. Отчёт OWASP Dependency-Check

Разделение SAST и SCA на два задания позволяет выполнять их параллельно и сразу понимать, связана ли ошибка с собственным кодом или со сторонней библиотекой.

8 Контрольные вопросы

8.1 1. Почему BCrypt предпочтительнее SHA-256 для паролей?

SHA-256 является быстрым универсальным криптографическим хешем. Скорость полезна для контроля целостности, но опасна для паролей: GPU и специализированное оборудование могут проверять огромное количество вариантов в секунду. Сам по себе SHA-256 также не задаёт обязательную соль и регулируемую стоимость.

BCrypt разработан специально для паролей. Он генерирует уникальную соль, включённую в итоговую строку, и имеет cost factor. Стоимость можно увеличивать по мере роста производительности оборудования. Поэтому каждая попытка перебора становится вычислительно дорогой, а готовые радужные таблицы теряют практический смысл.

8.2 2. В чём разница между SAST и DAST?

SAST анализирует исходный код, промежуточное представление или байткод без запуска приложения. Он удобен на ранних этапах, показывает конкретное место потенциальной проблемы и хорошо автоматизируется в pull request. Ограничение - ложные срабатывания и недостаток информации о реальной конфигурации выполнения.

DAST исследует работающее приложение как внешний клиент: отправляет запросы, изменяет параметры и оценивает ответы. Доступ к исходному коду не требуется. DAST показывает эксплуатационно достижимые проблемы, но хуже указывает их первопричину и не видит неисполненные ветви. На практике SAST и DAST дополняют друг друга.

8.3 3. Как работает JWT?

JWT состоит из трёх Base64URL-кодированных частей: Header, Payload и Signature. Header указывает тип и алгоритм. Payload содержит claims, но не шифруется, поэтому его нельзя считать секретным. Signature защищает токен от незаметного изменения.

Сервер формирует `base64url(header) + "." + base64url(payload)`, вычисляет подпись секретным или закрытым ключом и возвращает токен. При запросе сервер вычисляет подпись заново и сравнивает её с полученной, затем проверяет обязательные claims, например издателя и срок действия. В данном проекте используется HMAC-SHA-256, значит один секрет применяется для создания и проверки подписи.

8.4 4. Чем опасно отсутствие аудита зависимостей?

Сторонний компонент работает с правами приложения. Уязвимость в прямой или транзитивной зависимости может привести к удалённому выполнению кода, утечке данных, обходу аутентификации или отказу в обслуживании, даже если собственный код написан правильно.

Без SCA команда может продолжать использовать библиотеку с опубликованным CVE и готовым эксплойтом. Дополнительные риски: компрометация цепочки поставок, захват пакета, заброшенные версии и лицензионные ограничения. Регулярный автоматический аудит сокращает время между публикацией проблемы и обновлением зависимости.

9 Заключение

В ходе работы создан защищённый REST API на Java 21 и Spring Boot. Реализованы три метода, JWT-аутентификация, BCrypt-хеширование, параметризованный доступ к данным через JPA и экранирование пользовательского содержимого. Защищённые маршруты возвращают 401 без действующего токена.

Настроен GitHub Actions pipeline, запускающий восемь интеграционных тестов, SpotBugs и OWASP Dependency-Check на каждый push и pull request. Статический анализ помог выявить и устранить четыре замечания о работе с изменяемыми объектами. Итоговая сборка и SAST завершаются успешно.

10 Источники

1. OWASP Top 10. <https://owasp.org/www-project-top-ten/>.
2. OWASP Cheat Sheet Series. <https://cheatsheetseries.owasp.org/>.
3. JSON Web Token Introduction. <https://www.jwt.io/introduction>.
4. Spring Security Reference. <https://docs.spring.io/spring-security/reference/>.
5. GitHub Actions Documentation. <https://docs.github.com/actions>.
6. SpotBugs Maven Plugin. <https://spotbugs.github.io/spotbugs-maven-plugin/>.
7. OWASP Dependency-Check. <https://dependency-check.github.io/DependencyCheck/>.
8. Apache Logging Services Security. <https://logging.apache.org/security.html>.